Universidad Nacional de Costa Rica

Sede Regional Brunca, Campus Coto

EIF207 Estructuras de Datos, II Ciclo 2026

Proyecto II: Rutas aéreas

Estudiante: Freyman Jesus Guzmán Cortes

1. Introducción

Hice un planificador de vuelos en Java con NetBeans y Swing. Cada aeropuerto es un vértice de un grafo dirigido y ponderado, y cada ruta es una arista con dos pesos: minutos y costo. El programa encuentra la ruta con menos escalas (BFS), la más rápida y la más barata (Dijkstra) y los destinos alcanzables (DFS), y permite cancelar y restaurar rutas. Los datos se leen de los archivos, no están escritos en el código. Organicé el código en tres paquetes: modelo (grafo y carga), algoritmos e interfaz.



2. Estructuras de datos

Usé una lista de adyacencia: cada aeropuerto tiene una lista con las rutas que salen de él. La elegí porque hay pocas rutas por aeropuerto, entonces solo guardo las que existen. Un HashMap traduce el código del aeropuerto (por ejemplo, SJO) a su posición, y si el código no existe devuelve -1 para avisar sin que el programa se caiga.

BFS usa una cola, que recorre el grafo por niveles. Dijkstra usa una cola de prioridad, que siempre entrega el aeropuerto con menor distancia acumulada. DFS usa recursión. Hay un solo Dijkstra para los dos criterios: recibe si debe sumar minutos o costo. Para cancelar una ruta la quito de la lista y la guardo aparte, para poder restaurarla después.

3. Complejidad

Con n aeropuertos y m rutas: cargar los archivos cuesta O(n + m); buscar un aeropuerto, O(1) en promedio; BFS y DFS, O(n + m); y Dijkstra con cola de prioridad, O((n + m) log n). Cancelar una ruta solo recorre la lista de su origen.

4. Preguntas de análisis

Pregunta 1

DEN a SJO: menos escalas DEN-LAX-SJO (2 vuelos, 500 min, $895); más rápida DEN-DFW-IAHSJO (3 vuelos, 385 min, $525); más barata DEN-DFW-SAL-MGA-SJO (4 vuelos, 425 min, $435).

LIR a SJU: menos escalas LIR-JFK-SJU (2 vuelos, 510 min, $585); más rápida LIR-SJO-PTY-CCSSJU (4 vuelos, 355 min, $520); más barata LIR-SAL-MIA-SJU (3 vuelos, 385 min, $405).

SJO a MVD: menos escalas SJO-MIA-BSB-GRU-MVD (4 vuelos, 870 min, $1125); más rápida SJOPTY-LIM-EZE-MVD (4 vuelos, 595 min, $685); más barata SJO-PTY-LIM-GRU-MVD (4 vuelos, 720 min, $640).

La ruta de menos escalas no es la más rápida porque BFS cuenta vuelos y no su duración. Por ejemplo, LAX a SJO dura 355 minutos. La más rápida no es la más barata porque los vuelos rápidos suelen costar más y las conexiones baratas agregan escalas.

Pregunta 2

BFS no suma minutos, solo cuenta vuelos. Dijkstra funciona porque ningún peso es negativo, así que cuando fija la distancia de un aeropuerto ningún camino posterior la puede mejorar. Con un costo negativo podría dar una respuesta equivocada; en ese caso habría que usar Bellman-Ford.

Pregunta 3

Desde SJO no se llega a KIN, porque ninguna ruta llega a Kingston. ASU y MVD no pueden llegar a SJO, porque solo tienen rutas entre ellos. Lo primero lo calculo con un DFS desde SJO; lo segundo, con un DFS desde SJO sobre el grafo con las flechas invertidas.

Pregunta 4

Al cancelar LIM a CUZ, CUZ queda inalcanzable, porque es la única ruta que llega ahí. La encontré probando las 202 rutas: cancelaba cada una, hacía un DFS desde SJO y la restauraba. Es la única que provoca ese efecto.



Pregunta 5

Con estos datos, una matriz necesitaría 1600 casillas (40 × 40), casi todas vacías, y la lista unas 242 entradas. Con 4000 aeropuertos y 20 rutas cada uno, la matriz necesitaría 16 millones de casillas y la lista unas 84 mil entradas.

Pregunta 6

BFS y DFS cuestan O(n + m), y Dijkstra O((n + m) log n). Sin cola de prioridad, Dijkstra tendría que buscar el mínimo recorriendo todos los aeropuertos en cada paso, lo que da $O(n^2)$.

5. Herramientas

Usé un asistente de IA (Claude) como apoyo durante el desarrollo, como permite el enunciado. El fondo del mapa viene de Natural Earth, de uso libre, y es solo decorativo.